

2022-9-1

文档编号		版本	1.0
文档名称	weston 介绍		
项目来源			

weston 介绍

编写：日期： 2026-9-1

检查：日期：

审核：日期：

批准：日期：

[illegible]

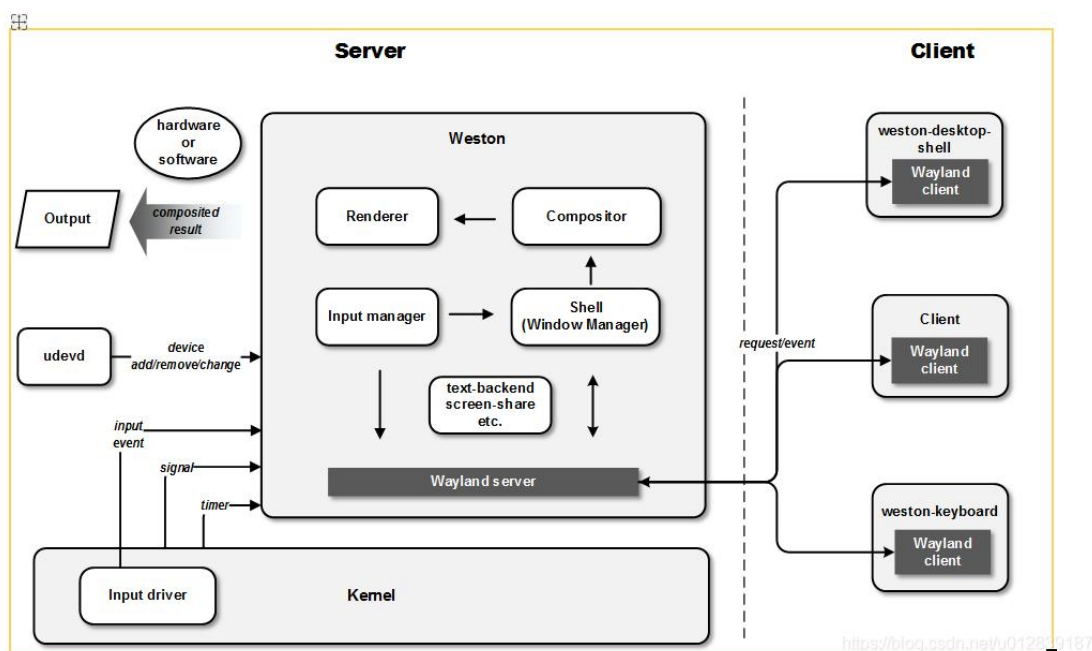
目录

1 wayland&weston 介绍.....	3
2 server 和 client 之间通讯.....	4
3 Weston.ini 配置文件.....	5
4 Client 渲染流程.....	5
5 Weston 渲染流程.....	6
6 IVI-shell.....	7
6.1 概述.....	7
6.2 配置.....	9
6.3 使用.....	9
6.4 pvtsoft-layer-surfaces 配置.....	11
7 BootAnimation.....	13
8 Debug.....	14
9 Weston 异常.....	15
10 FAQ.....	16

1 wayland&weston 介绍

Wayland 基于 domain socket 实现了一套 display server 与 client 间通信的库，并且以 XML 形式定义了一套可扩展通信协议。Weston 属于 server 部分。

Weston 从内部体系结构来看，主要分为窗口管理 (shell)，合成器 (compositor) 和输入管理等几个部分。



1、compositor backend 主要决定了 compositor 合成完后的结果怎么处置。从数据结构上，weston_output 是 output 设备的抽象，而下面的 backend 会实现具体的 output 设备。

- fbdev: 直接输出至 linux 的 framebuffer 设备。接口通用。
- DRM: Direct rendering manager，桌面上一般用这个。
- x11: Wayland compositor 作为 X server 的 client。它可以让 Wayland client 运行在 X11 上。
- wayland: Wayland compositor 作为 server 同时，也作为另一个 Wayland compositor 的 client。用于 nested compositor。

2、renderer backend 主要用于 compositor 的合成之用，除去 noop-renderer 外，有 gl-renderer 和 pixman-renderer 两种。前者为 GPU 硬件渲染，后者为软件渲染。

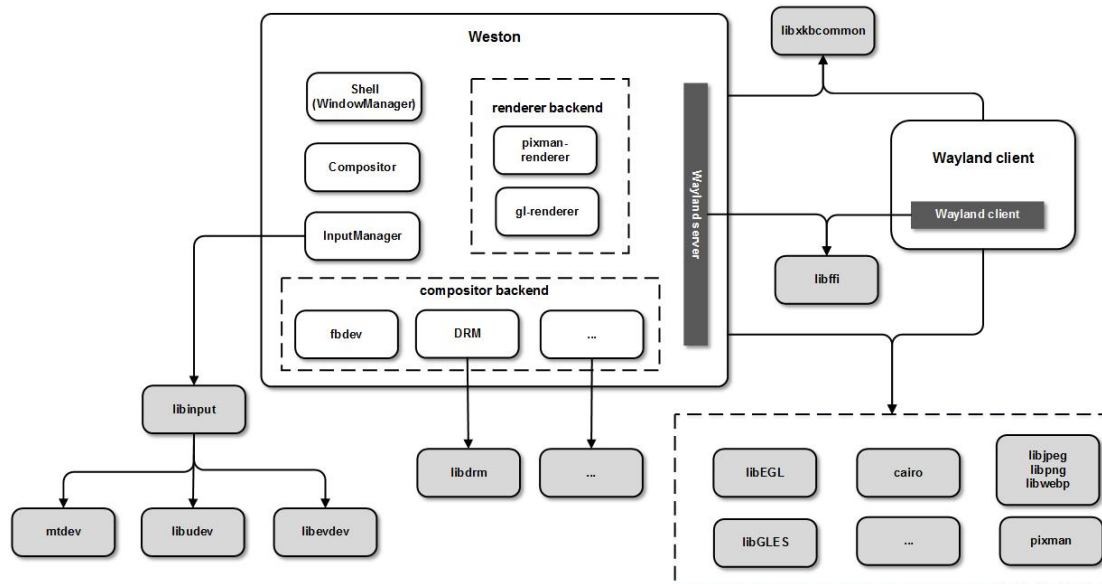
3、shell backend 用于实现具体的窗口管理。相应的实现分别在 desktop-shell, fullscreen-shell 和 ivi-shell 目录中。

4、input manager 用于对输入设备（键盘，鼠标，触摸屏等）的管理，提供了设备检测，设备处理，输入事件处理等基本功能。

- mtdev: Multi-touch 设备处理，比如它会将不带 tracking ID 的 protocol A 转化为 protocol B。

- libevdev: 处理 kernel 中 evdev 模块对接。
- libudev: 主要用于和 udevd 的通信, 从而获取设备的增加删除事件。也可从 kernel 获取。

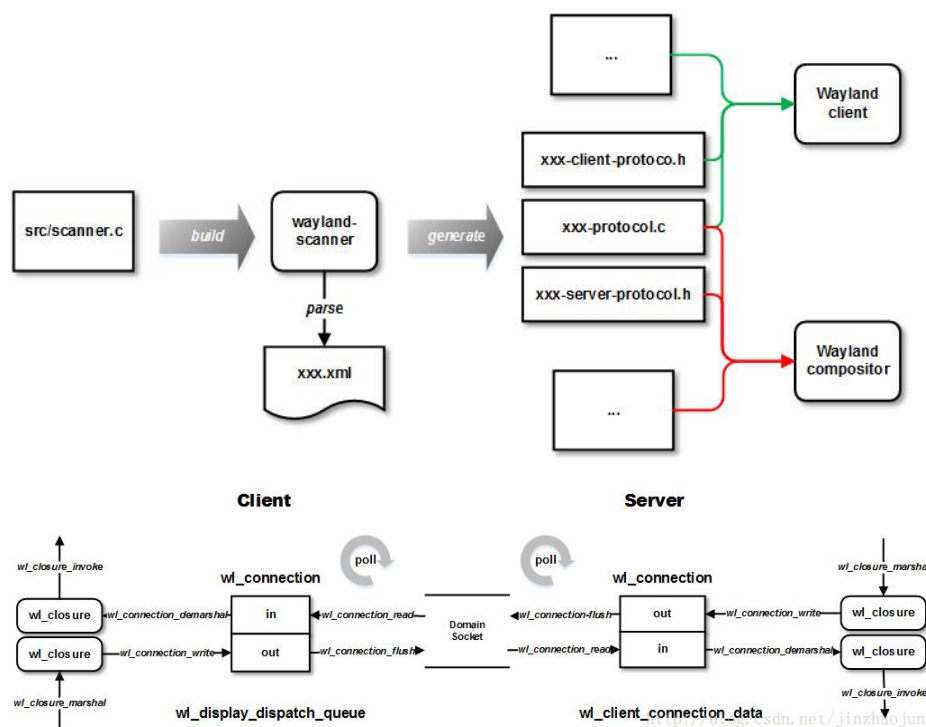
Wayland/Weston 运行时的依赖主要有下面几个, 其相互关系大体如下。



- libEGL, libGLES: 本地窗口系统与图形 driver 的 glue layer, mesa 提供了开源版本的实现。底层操作 GPU
 - libdrm: 封装 KMS, GEM 等图形相关接口。Display 平台相关。
 - libffi: 用于在运行时根据调用接口描述生成函数跳板并调用。
 - pixman: 用于像素操作的库, 包括 region, box 等计算。用了许多平台相关的优化。
 - cairo: 软件渲染库, 类似于 skia。也有 OpenGL 后端。
 - libinput: 输入处理, 依赖于 mtdev, libudev, libevdev 等库。
 - libxkbcommon: 主要用于键盘处理。
 - libjpeg, libpng, libwebp: 用于加载各种图片文件, 像壁纸, 面板和鼠标等都需要。
- 可见 weston 主要依赖两大硬件 display 和 GPU。初始化也与之息息相关。

参考: <https://zhuanlan.zhihu.com/p/690561669>

2 server 和 client 之间通讯



Wayland 中把 Client 发给 Server 的跨进程函数调用称为 request，反方向的跨进程函数调用称为 event。首先通过 libffi 库将调用抽象成数据流的形式，然后这些信息通过 socket 进行传输。

参考：<https://zhuanlan.zhihu.com/p/434869637>

3 Weston.ini 配置文件

<https://zhuanlan.zhihu.com/p/396168706>

4 Client 渲染流程

```
wl_display_connect(NULL);  连接 Server，绑定服务

wl_registry_add_listener(display->registry,&registry_listener, display); //监听远程服务对象的
本地代理对象 wl_compositor、xdg_wm_base、zwp_fullscreen_shell_v1、wl_shm、
ivi_application、wl_seat

wl_compositor_create_surface(display->compositor);
ivi_application_surface_create(window->ivi_application,
                                id_surface, window->wlBkgndSurface); //ivi-shell 创建 surface
//xdg_wm_base_get_xdg_surface(display->wm_base,
                                window->wl_surface); //desktop-Shell 创建 surface
```

```
pool = wl_shm_create_pool(window->wl_shm, fd, size);
window->wlBkgndBuffer = wl_shm_pool_create_buffer(pool,
    0,width,height,stride,WL_SHM_FORMAT_ARGB8888);    //分配共享 buffer

/*绘制*/
wl_surface_attach(window->wlBkgndSurface, window->wlBkgndBuffer, 0, 0);    //拿到 buffer
wl_surface_damage(window->wlBkgndSurface, 0, 0, width, height);    //计算需要重绘区域
wl_surface_frame(window->wlBkgndSurface);
wl_callback_add_listener(window->callback, &frame_listener, window);    //设置绘制完成回调
wl_surface_commit(window->wlBkgndSurface);    //提交绘制请求

while (ret != -1)
    ret = wl_display_dispatch(window->wl_display);    //监测信息
```

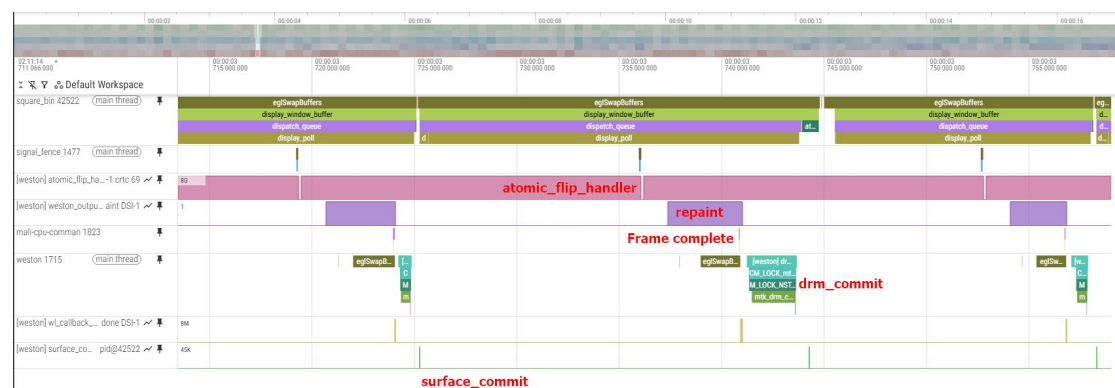
从 code 看 Client 的 surface 主要的两种创建方式:

- 1、 Desktop Shell: 画面布局是固定的, 包括 title, panle,backgroud 等, 不能满足需求。
- 2、 IVI Shell(ivi-controller):是 Covesa(以前 genivi)组织提供的开源代码 wayland ivi extension, 启动后只是提供了一块黑色屏幕, 编译后生成了基于 ivi_wm 协议的 ilm api(以 so 库方式存在), 提供 layer 和 surface 概念便于实现层级管理。

Client 的几种类型:



5 Weston 渲染流程



Client 通过 surface_commit 提交一帧。等待上一帧完成，display 触发 atomic_flip_handler,weston 执行 repaint，将所有的 surface 通过 GPU 合成，并调用 eglSwapBuffers（等 Frame complete 信号）。紧接着调用 drm_commit 送到 display。

参考

<https://blog.csdn.net/u012839187/article/details/100580627>

<https://blog.csdn.net/u012839187/article/details/106469038>

6 IVI-shell

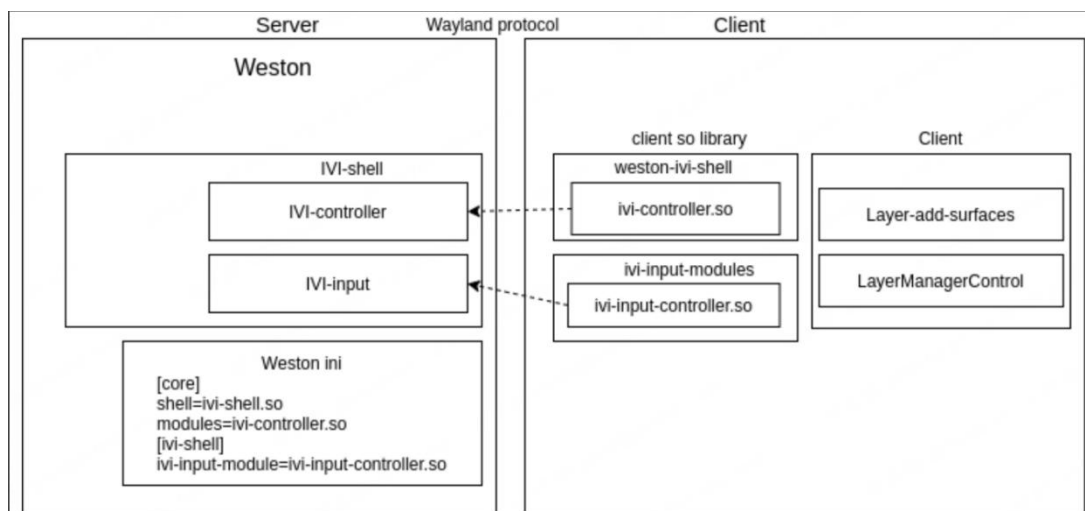
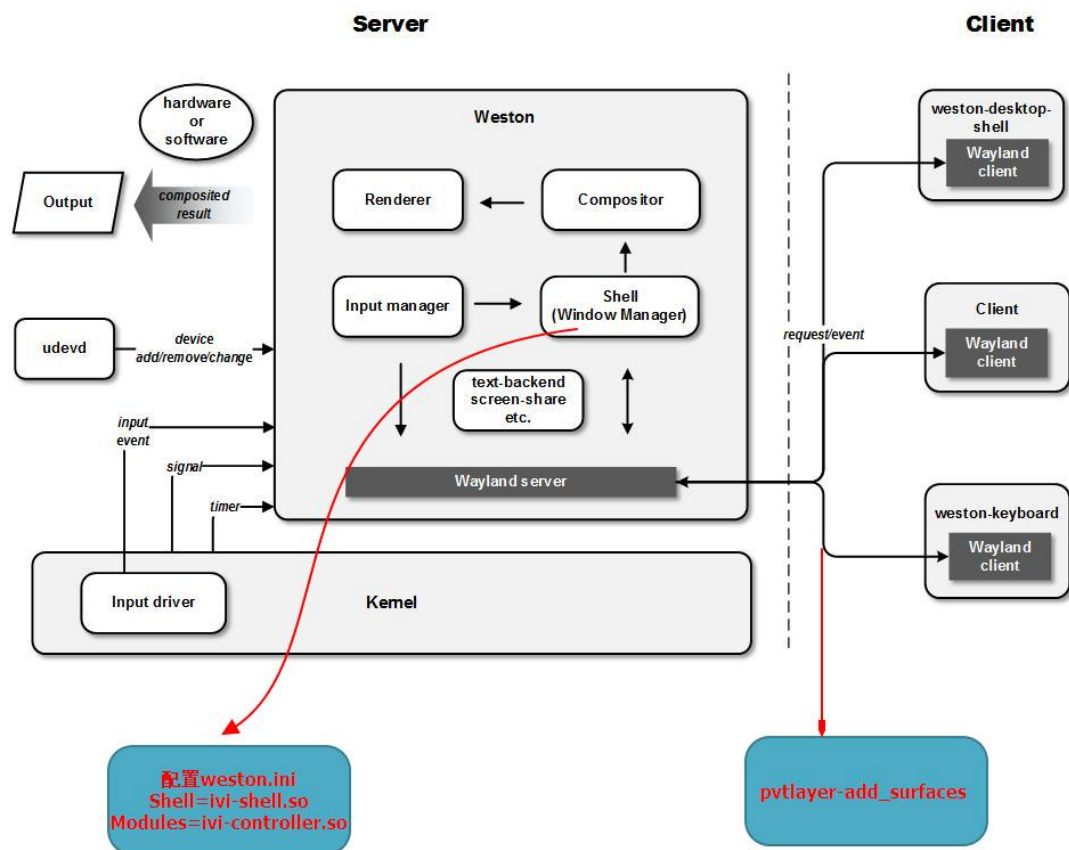
6.1 概述

当前 weston 有三个窗口管理 shell：Desktop-shell, Fullscreen-shell 和 IVI-shell，每个 shell 都有自己的公共的客户端协议接口。客户端程序必须按照一种 shell 协议编写客户端，否则它将无法正常工作。

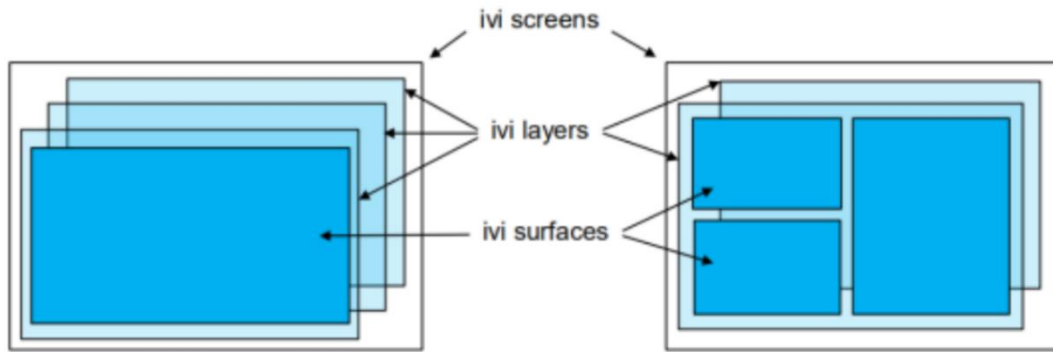
- Desktop Shell: 像现代 X 桌面环境一样，专注于传统的键盘和鼠标用户界面以及熟悉的类似桌面的窗口管理。Desktop-shell 由 desktop-shell.so 和 weston-desktop-shell 插件组成。
- Fullscreen Shell: fullscreen-shell 适用于客户端想要接管所有的输出的场景。客户端在 weston 运行一个 compositor。另外一个 compositor 不需要处理任何特定于平台的内容例如 DRM/KMS 或 evdev/libinput。fullscreen-shell 仅由 fullscreen-shell.so 插件组成。
- IVI shell : IVI (in-vehicle infotainment) , IVI-shell 是一种专用的 shell，它向控制模块提供了 GENIVI 层管理兼容的 API，对客户端来讲，这是一种简单的协议。IVI-shell 首先加载 ivishell.so 插件，然后加载 ivi-controller.so 插件。

针对上述三种 shell，Desktop-shell 最常用，适用于传统的键盘和鼠标的用户界面管理。Fullscreen Shell 比较少用，适用于客户端想要接管所有的输出的场景，它的一个实际用例是屏幕录制和共享。IVI-Shell 由于对客户端提供了简单的控制接口，从而使用者可以方便地对图层及其显示内容的上下层关系，透明度及位置关系等进行控制。为多窗口的应用开发带来了便捷性。

因此，在汽车 IVI 和 ADAS 应用中，客户普遍采用了 IVI-shell。



主要的配置文件是 weston.ini，其中 shell 环境配置为 ivi-shell.so；modules 配置为 ivi-controller.so；ivi-input-module 配置为 ivi-input-controller.so。客户端利用 layer-add_surfaces 和 LayerManagerControl 对图层和 surface 进行配置。当 weston 配置为 ivi-shell 后，使用 ivi 应用协议去管理客户端程序。Wayland-ivi-extension 提供了 ivi-controller.so 插件去配置 surfaces/layers/screens 的属性，也提供了 ivi-input-controller.so 插件去管理输入事件。



首先，surface 代表 wayland client 的一个绘图表面。client 通过画好的 buffer attach 到 surface 上来更新窗口，因此说 surface 是 buffer 的载体。在 weston 中，shell 是窗口管理器，因此一个 surface 要想被窗口管理器管理，需要创建相应的 shell surface。如图：在一个 IVI 屏幕上，我们首先要创建一个图层 layer，然后在图层 layer 上创建 surface 显示内容，在一个 layer 上可以创建多个 surface 来显示多个客户端应用。而 screen 代表一个屏幕，一个 screen 可以创建多个 layer，设置 z-order 可以设置对应的层级。

6.2 配置

weston.ini

```
[core]
shell=ivi-shell.so
require-input=false
modules=ivi-controller.so

[ivi-shell]
ivi-input-module=ivi-input-controller.so
ivi-id-agent-module=ivi-id-agent.so

[desktop-app-default]
default-surface-id=2000000 //默认 surface id, 没添加 surface id 加一
default-surface-id-max=2001000 //最大 2001000
```

6.3 使用

1、layer-add-surface 操作

layer-add-surface 用于创建 layer

```
#添加 layer id 1,允许绑定 1000 个 surface, 指定 DP-1 为输出端
layer-add-surface -l 1 -s 1000 -d DP-1 &
```

2、LayerManagerControl 的操作

LayerManagerControl 的接口可以方便地调整图层地缩放、位置、透明度、可见性等

```
#显示 surface 信息
LayerManagerControl get surface 2000000
surface 2000000 (0x1e8480)
-----
- created by pid:      2709    //进程 pid
- original size:      x=1926, y=753
- destination region: x=0, y=0, w=1926, h=753
- source region:      x=0, y=0, w=1926, h=753
- opacity:            1    //透明度
- visibility:         1
- frame counter:      56696
- on layer:           2(0x2) 1(0x1)

#显示 layer 信息
LayerManagerControl get layer 1
layer 1 (0x1)
-----
- destination region:  x=0, y=0, w=1920, h=1080
- source region:       x=0, y=0, w=1920, h=1080
- opacity:             1
- visibility:          1
- surface render order: 2000000(0x1e8480), 2000001(0x1e8481),
- on screen:           0(0x0)

#显示 screen 信息
LayerManagerControl get screen 0
screen 0 (0x0)
-----
- connector name:      DSI-1
- resolution:          x=1920, y=1080
- layer render order:  1(0x1), 6(0x6),

#创建 layer
LayerManagerControl create layer <layerid> <width> <height>
#销毁 layer
LayerManagerControl destroy layer <id>
#dump screen|surface 画面到文件
LayerManagerControl dump screen|surface <id> to <file>
#得到 scene|screens|layers|surfaces 信息
LayerManagerControl get scene|screens|layers|surfaces
#得到 screen|layer|surface 某一信息
LayerManagerControl get screen|layer|surface
```

```

#将 surface 添加到 layer
LayerManagerControl add surface <sid> to layer <lid>
#从 layer 删除 surface
LayerManagerControl remove surface <sid> from layer <lid>

#设置 layer|surface 的透明度 (0.0~1.0)
LayerManagerControl set layer|surface <id> opacity <opacity>
#设置 layer|surface 是否显示 (0: 不显示; 1: 显示)
LayerManagerControl set layer|surface <id> visibility <visibility>
#设置 screen|layer 与 layer|surface 链接
#如: LayerManagerControl set screen 0 render order 1,6 //数字顺序决定显示层次
LayerManagerControl set screen|layer <id> render order [<idarray>]

#显示 layer|surface 所有设置操作
LayerManagerControl watch layer|surface <idarray>
#测试 layer
LayerManagerControl test notification layer <layerid>

```

3.example

```

$ layer-add-surface 1 1000& 添加 surface
$ export XDG_RUNTIME_DIR=/var/run/user/1000
$ weston & //打开 weston
$ LayerManagerControl create layer 1000 600 400 //创建 layer; id 为 1000. 宽 600.长 400
$ LayerManagerControl set layer 1000 render order 10 //为 layer1000 链接上 id 为 10 的 surface
$ LayerManagerControl set surface 10 source region 0 0 600 400 //将 surface10 的内容从 (0,0)开始截取一个 600*400 的范围
$ LayerManagerControl set surface 10 destination region 0 0 600 400 //将 surface 截取后的内容放置到 layer(0,0)的位置, 缩放为 600*400 的大小
$ LayerManagerControl set screen 0 render order 1000 //将 screen0 与 layer1000 链接
$ LayerManagerControl set layer 1000 visibility 1 //将 layer1000 设置为可见
$ LayerManagerControl set surface 10 visibility 1 //将 surface 设置为可见

```

6.4 pvtsoft-layer-surfaces 配置

通过配置文件/vendor/etc/hyper_android/ivi_shell_cfg.pb.txt 的方式, 方便配置 UI 应用的显示信息: 显示大小、尺寸、位置和层级。

```
display_info { #第一个显示屏
```

```

connector_name: "DP-1"           #必要参数，用于匹配 connector_name

screen_width: 1920,              #非必要，从 weston 获取屏参
screen_height: 720,              #非必要，从 weston 获取屏参
layer_count: 6,                  #非必要，从下面 layers 个数获取
layers {                          #第一个 layer
    layer_id: 90500,              #必要参数，用于创建 layer
    opacity: 1.0,                 #必要参数，设置 layer 透明度
    sourceX: 0,                   #可选，截取 layer 显示区域 x，不填为 0
    sourceY: 0,                   #可选，截取 layer 显示区域 y，不填为 0
    source_width: 1920,           #可选，截取 layer 显示区域 w，不填为 screen_width
    source_height: 720,           #可选，截取 layer 显示区域 h，不填为 screen_height
    destX: 0,                     #可选，设置 layer 显示起始位置 x，不填为 0
    destY: 0,                     #可选，设置 layer 显示起始位置 y，不填为 0
    dest_width: 1920,             #可选，设置 layer 显示起始位置 w，不填为 screen_width
    dest_height: 720,             #可选，设置 layer 显示起始位置 h，不填为 screen_height
    visibility: true,             #必要参数，设置 layer 是否显示
    surface_count: 1,             #非必要，从下面 surfaces 个数获取
    surfaces {                    #layer: 90500 下的第一个 surface
        surface_id: 90500500,     #必要参数，用于识别 surface，大于 0 时，用于识别 ivi_surface client
        surface_name: "hmi",      #可选，当 surface_id 为 0 时，用于识别 xdg_surface client
        opacity: 1.0,             #必要参数，设置 surface 透明度
        sourceX: 0,               #可选，截取 surface 显示区域 x，不填为 0
        sourceY: 0,               #可选，截取 surface 显示区域 y，不填为 0
        source_width: 1920,        #可选，截取 surface 显示区域 w，不填为 client width
        source_height: 720,        #可选，截取 surface 显示区域 h，不填为 client height
        destX: 0,                 #可选，设置 surface 显示起始位置 x，不填为 0
        destY: 0,                 #可选，设置 surface 显示起始位置 y，不填为 0
        dest_width: 1920,          #可选，设置 surface 显示起始位置 w，不填为 client width
        dest_height: 720,          #可选，设置 surface 显示起始位置 h，不填为 client height
        visibility: true,          #必要参数，设置 surface 是否显示
        fullscreen: true,          #可选，设置全屏显示
    }
    surfaces {                    #layer: 90500 下的第二个 surface，层级是往下越高，即显示在上层
        surface_id: 90500501,
        .....
    }
}

layers {                          #DP-1 下的第二个 layer，层级是往下越高，即显示在上层
    layer_id: 90700,
    .....
    surfaces {
        surface_id: 90700700,
        .....
    }
}

```

```

    }
}

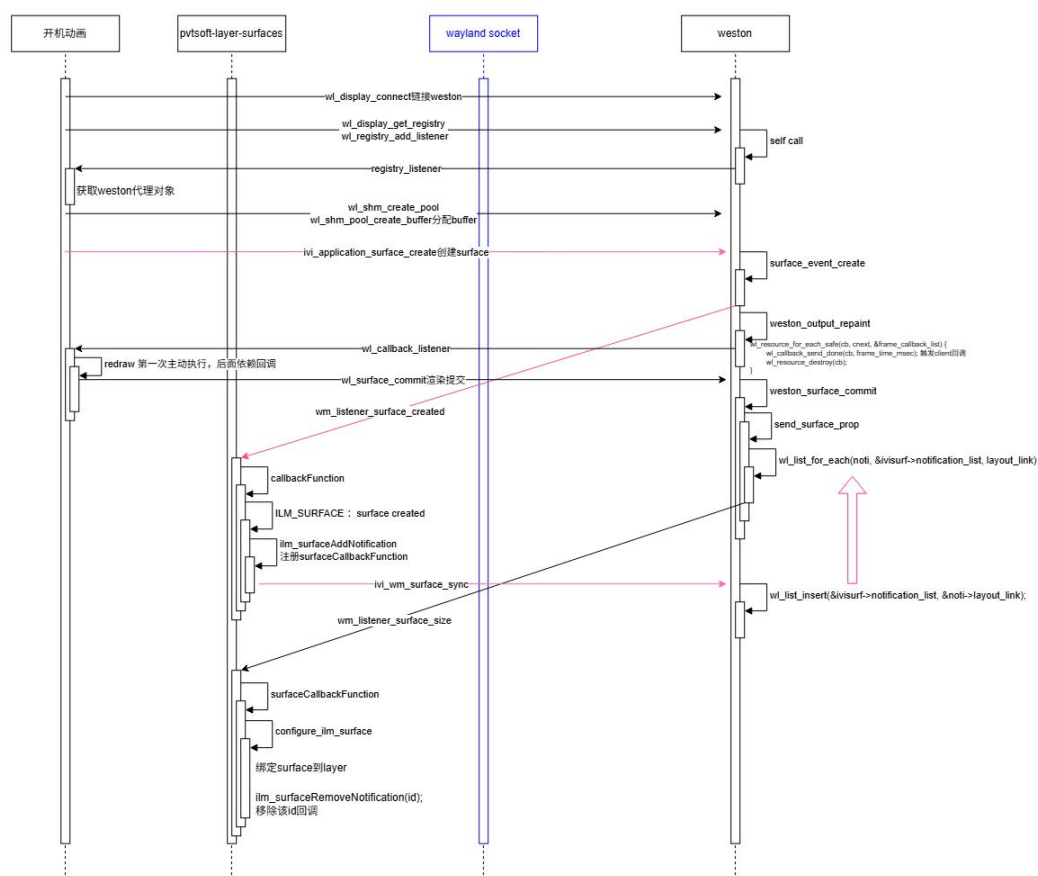
display_info {
    connector_name: "DSI-1"
    .....
}

```

ivi_shell_cfg.pb.txt 配置说明:

- 1、动态分配 layer， 一个 screen 上添加几组 layer， 就分配几组 layer， 且 layer 组往后定义的层级越高。
- 2、同一 layer 下可以绑定多个 surface， surface 组往后定义的层级越高。但只能定义 ivi_surface。需要通过 surface_id 来设置层级。
- 3、id 可以定义十进制和十六进制 (0x)
- 4、client 可以是 ivi_surface 和 xdg_surface。通过 surface_id 是否为 0 区分。
- 5、同一 surface 可以和多个 layer 绑定， 实现多显。

7 BootAnimation



BootAnimation 调用 `ivi_application_surface_create` 创建, weston 会触发 `layer-add_surfaces` 回调 `callackFunction`。但 BootAnimation 调用 `wl_surface_commit` 后, weston 会触发 `layer-add_surfaces` 回调 `surfaceCallbackFunction` 进行 `surface` 跟 `layer` 绑定。同时 weston 执行 `weston_output_repaint` 进行渲染, 会通过 `wl_callback_send_done` 将回调发给 BootAnimation

8 Debug

`weston.service` 中加入“`--log=/tmp/weston.log --debug`”打开调试
weston 启动 log 保存在 `/tmp/weston.log`

shell 执行:

```
weston-debug -a //显示所有调试信息
weston-screenshooter //截屏
weston-simple-egl //显示 demo fps
weston-simple-touch //显示触摸轨迹
```

1、weston 状态

```
# service 状态
systemctl status weston
# weston log
journalctl -xeu weston
weston-debug -a //先执行 source /data/wayland_env_file
# drm active (weston wake)
cat /sys/class/drm/card0/device/power/runtime_status
# weston 使用的句柄
ls /proc/$(echo $(pidof weston) | cut -d ' ' -f 1)/fd
ls /proc/$(echo $(pidof weston) | cut -d ' ' -f 2)/fd
# weston 进程状态
ps elf $(echo $(pidof weston) | cut -d ' ' -f 1)
ps elf $(echo $(pidof weston) | cut -d ' ' -f 2)
```

2、weston dump

```
# 以下先执行 source /data/wayland_env_file
# 所有屏幕截图, weston.service 加--debug 使用
weston-screenshooter
# dump 每个屏幕的 buffer
LayerManagerControl get screens //得到 id
LayerManagerControl dump screen <id> to screen_num.png

# dump 每个 surface 的 buffer
LayerManagerControl get surfaces //得到 id
LayerManagerControl dump surface <id> to surface_num.png
```

3、weston 显示 demo

```
# 以下先执行 source /data/wayland_env_file
# DSI-1 是哪个屏幕, weston-debug scene-graph 查看,或
# LayerManagerControl get screens && LayerManagerControl get screen <id>
layer-add-surfaces -l 1 -s 1000 -d DSI-1
# 另外的窗口输入
# egl 显示帧率
weston-simple-egl
# surface_commit 显示
weston-simple-shm
# 测试 touch
weston-simple-touch
# gstreamer 显示视频
gst-launch-1.0 playbin flags=0x42 video-sink="v4l2convert output-io-mode=dmabuf-
import capture-io-mode=dmabuf disable-passthrough=true propose-allocation=3
name=mtkmdp ! video/x-raw,format=BGRA,width=640,height=360,pixel-aspect-ratio=1/1 !
waylandsink" uri=file:///app/res/output.h264
```

```
gst-launch-1.0 playbin flags=0x42 video-sink="v4l2convert output-io-mode=dmabuf-
import capture-io-mode=dmabuf disable-passthrough=true propose-allocation=3
name=mtkmdp ! video/x-raw,format=BGRA,width=940,height=300,pixel-aspect-ratio=1/1 !
waylandsink surface-id=90500500" audio-sink="audioconvert ! audioresample ! audio/x-
raw,rate=48000,format=S16LE ! alsasink device=hw:0,6" uri=file:///usr/animation/4k.mp4
```

9 Weston 异常

1、

Weston 异常一般跟 display 和 GPU 有关。如果是硬件问题,一般只能重启车机才有可能恢复。不过不建议这样做,需要保留现场确定原因

(1) GPU hang 导致 weston 卡住

GPU 遇到 hang 如 fence timeout 时有自己的 recovery 机制,例如 force signaled, reset gpu, 不过 reset 并不是万能的,一些问题需要具体分析原因。也可以手动复位: echo 1 > /sys/kernel/debug/mali0/reset

总的来说,发生异常时不需要主动去 echo reset, GPU driver 会自己做 error handle

(2) drm 无法正常 flip

flip 失败有多种原因,原因无法一一罗列。时序问题导致 flip 失败,可以重启 weston 解决。硬件问题导致 flip 失败,只能重启车机了。

10 FAQ

1, 补充说明梳理依赖关系: weston 依赖哪些? 哪些依赖 weston? 简要说明依赖逻辑。

答: 被 weston 依赖: gpu (egl、gl)、display (drm)、input

主要依赖的硬件模块, 跟 weston 运行相关。

依赖 weston: gpu_server

gpu server 初始化会准备 opengl egl 环境, 这些是 mali so 加载, mali 初始化 egl 的时候会尝试连接 weston, 连不上就会出问题。

2, 补充说明 weston server crash 后各 client 的最佳实践?

答: client 主流的做法都是通过 wl_display_dispatch 检测与 server 连接状态
weston demo

```
build-sos > tmp > work > aarch64-poky-linux > weston > 13.0.0 > weston-13.0.0 > clients > C simple-shm.c
686  {
703  wl_surface_damage(window->surface, 0, 0,
705
706      if (!window->wait_for_configure)
707          redraw(window, NULL, 0);
708
709      while (running && ret != -1)
710          ret = wl_display_dispatch(display->display);
711                                     disconnect 返回 EPIPE, 退出loop
712      fprintf(stderr, "simple-shm exiting\n");
713  }
```

egl 也是一样

```
build-sos > tmp > work > aarch64-poky-linux > weston > 13.0.0 > weston-13.0.0 > clients > C simple-egl.c
1305  {
1388      sigint.sa_flags = SA_RESETHAND;
1389      sigaction(SIGINT, &sigint, NULL);
1390
1391      while (running && ret != -1) {
1392          ret = wl_display_dispatch_pending(display->display);
1393          redraw(&window);
1394      }
1395
1396      fprintf(stderr, "simple-egl exiting\n");
1397  }
```

QT 也是如此

```

build-sos > tmp > work > aarch64-poky-linux > qtwayland > 5.15.13+git > git > src > client > qwaylanddisplay.cpp
219 {
220
221 }
222
223 void QWaylandDisplay::blockingReadEvents()
224 {
225     if (wl_display_dispatch(mDisplay) < 0)
226         checkError();
227 }
228
229 void QWaylandDisplay::checkError() const
230 {
231     int ecode = wl_display_get_error(mDisplay);
232     if ((ecode == EPIPE || ecode == ECONNRESET)) {
233         // special case this to provide a nicer error
234         qWarning("The Wayland connection broke. Did the Wayland compositor die?");
235     } else {
236         qWarning("The Wayland connection experienced a fatal error: %s", strerror(ecode));
237     }
238     _exit(1);
239 }

```

disconnect 返回 EPIPE，退出 loop。

完整获取错误码

```

while (running && ret != -1) {
    ret = wl_display_dispatch_pending(display.display);
    if (ret == -1) {
        int err = wl_display_get_error(display.display);
        fprintf(stderr, "ret == -1 err=%d\n", err);
        if (err == EPROTO) {
            const struct wl_interface *interface;
            uint32_t object_id;
            uint32_t code = wl_display_get_protocol_error(display.display, &interface, &object_id);
            fprintf(stderr, "Protocol error: %s, object %u, code %u\n",
                    interface->name, object_id, code);
        } else if (err == ENOMEM) {
            fprintf(stderr, "Out of memory\n");
        } else if (err == EPIPE) {
            fprintf(stderr, "Connection lost\n");
        } else if (err == EINVAL) {
            fprintf(stderr, "Invalid argument\n");
        }
    }
    redraw(&window);
}

```

3，如果显示出现问题，weston 是否支持热重启，热重启包含哪些组件或进程，pvtsoft_layer_surface 是否需要重启？

答：跟 weston 运行息息相关的模块（GPU 和 drm）都会导致 weston 重启，因为 weston 调用 drm、egl 和 gl 接口，出现异常会返回错误码。进行相应处理。pvtsoft_layer_surface 是 weston 一个 client，会随 weston 重启而重启。重启过程中只重启 weston 和 client，weston 重启会初始化 egl、gl 和 drm 等组件。

4、设置背景透明

/etc/xdg/weston/weston.ini

background-transparent=DP-1,DSI-1

